

Lisp typé

Rawšanī

1. Introduction

Typer Lisp est un cas particulier, et non le plus simple, du problème de typer un langage homoïconique. La philosophie et la syntaxe de Lisp interdisent la plupart des solutions déjà avancées pour le λ -calcul homoïconique — en particulier tout recours aux types algébriques généralisés, que Lisp ne permet pas de définir.

2. Définitions

Contextes. Un contexte Γ est une suite finie d'associations symbole/type :

$$\Gamma ::= \emptyset \mid \Gamma, x : \tau$$

d'appartenance dérivée par :

$$\frac{}{x \in (\Gamma, x : \tau)} \text{ HERE}$$
$$\frac{x \in \Gamma}{x \in (\Gamma, y : \sigma)} \text{ THERE}$$

Symboles. Chaque symbole possède son propre type singleton, à raison d'un habitant par symbole : **Sym 'a**, **Sym 'b**, **Sym 'f**, etc. Un habitant de **Sym 'f** ne peut être que 'f — ainsi un terme porte-t-il l'identité exacte du symbole qu'il référence.

Code. Code $\Gamma \tau$ est builtin, inspectable par **car** et **cdr**.

3. Type System

Les règles **HERE** et **THERE**, ci-dessus, définissent l'appartenance d'une variable

à un contexte. On en déduit directement la règle de typage des variables :

$$\frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau} \text{ VAR}$$

Les abstractions étendent le contexte avec leur paramètre :

$$\frac{\Gamma, x : \tau \vdash e : \sigma}{\Gamma \vdash (\text{lambda } x e) : \tau \rightarrow \sigma} \text{ ABS}$$

et les applications imposent la compatibilité entre le type de la fonction et celui de son argument :

$$\frac{\Gamma \vdash f : \tau \rightarrow \sigma \quad \Gamma \vdash e : \tau}{\Gamma \vdash (f e) : \sigma} \text{ APP}$$

Les littéraux entiers et booléens sont typés sans condition sur Γ :

$$\frac{n \in \mathbb{Z}}{\Gamma \vdash n : \mathbb{Z}} \text{ INT}$$

$$\frac{c \in \{\top, \perp\}}{\Gamma \vdash c : \mathbb{B}} \text{ BOOL}$$

let introduit la variable typée dans le contexte de son corps :

$$\frac{\Gamma \vdash e_1 : \sigma \quad \Gamma, x : \sigma \vdash e_2 : \tau}{\Gamma \vdash (\text{let } (x e_1) e_2) : \tau} \text{ LET}$$

Deux constructeurs **cons** sont nécessaires, l'un hétérogène, l'autre homogène récursif — Lisp ne distingue pas syntaxiquement la paire de la liste, le typeur, lui, y est contraint :

$$\frac{\Gamma \vdash x : \tau \quad \Gamma \vdash y : \sigma}{\Gamma \vdash (\text{cons } x y) : \tau \times \sigma} \text{ CONS}$$

$$\frac{\Gamma \vdash x : \tau \quad \Gamma \vdash xs : \mathbf{List} \tau}{\Gamma \vdash (\mathbf{cons} \ x \ xs) : \mathbf{List} \tau} \text{LCONS}$$

La liste vide est polymorphe :

$$\frac{}{\Gamma \vdash () : \mathbf{List} \tau} \text{NIL}$$

`car` et `cdr` sont uniformément typés sur les paires, les listes, et `Code` — trois instances de la même idée, un accès à la première ou seconde composante d'un produit :

$$\frac{\Gamma \vdash p : \tau \times \sigma}{\Gamma \vdash (\mathbf{car} \ p) : \tau} \text{CARPAIR}$$

$$\frac{\Gamma \vdash p : \tau \times \sigma}{\Gamma \vdash (\mathbf{cdr} \ p) : \sigma} \text{CDRPAIR}$$

$$\frac{\Gamma \vdash l : \mathbf{List} \tau}{\Gamma \vdash (\mathbf{car} \ l) : \tau} \text{CARLIST}$$

$$\frac{\Gamma \vdash l : \mathbf{List} \tau}{\Gamma \vdash (\mathbf{cdr} \ l) : \mathbf{List} \tau} \text{CDRLIST}$$

$$\frac{\Gamma \vdash q : \mathbf{Code} \ \Gamma \ (\tau \times \sigma)}{\Gamma \vdash '(\mathbf{car} \ ,q) : \mathbf{Code} \ \Gamma \ \tau} \text{CARCODE}$$

$$\frac{\Gamma \vdash q : \mathbf{Code} \ \Gamma \ (\tau \times \sigma)}{\Gamma \vdash '(\mathbf{cdr} \ ,q) : \mathbf{Code} \ \Gamma \ \sigma} \text{CDRCODE}$$

L'instanciation permet de passer d'un type polymorphe à une instance — nécessaire, en particulier, pour spécialiser le τ demeuré libre de `NIL` :

$$\frac{\Gamma \vdash e : \sigma \quad \sigma \sqsubseteq \sigma'}{\Gamma \vdash e : \sigma'} \text{INST}$$

La généralisation est autorisée lorsque la variable de type n'apparaît pas librement dans le contexte :

$$\frac{\Gamma \vdash e : \sigma \quad \alpha \notin \text{Free}(\Gamma)}{\Gamma \vdash e : \forall \alpha. \sigma} \text{GEN}$$

`QUOTE` recense dans Γ les symboles libres du terme cité, indépendamment du contexte ambiant Δ où la citation est écrite :

$$\frac{\Gamma \vdash e : \tau}{\Delta \vdash (\mathbf{quote} \ e) : \mathbf{Code} \ \Gamma \ \tau} \text{QUOTE}$$

Un terme clos n'a nul besoin d'attendre un environnement : sa citation se résout immédiatement.

$$\frac{\emptyset \vdash e : \tau}{\Delta \vdash (\mathbf{quote} \ e) : \tau} \text{QCONST}$$

Une fonction, déjà en attente d'application, n'est pas altérée par sa citation.

$$\frac{\Gamma \vdash f : \tau \rightarrow \sigma}{\Gamma \vdash (\mathbf{quote} \ f) : \tau \rightarrow \sigma} \text{QABS}$$

La paire citée n'est jamais elle-même du `Code`, mais un couple ordinaire de deux termes `Code` — car rien ne garantit que chacune de ses composantes soit indépendamment évaluable sans le reste de son environnement.

$$\frac{\Gamma \vdash a : \tau \quad \Gamma \vdash b : \sigma}{\Delta \vdash (\mathbf{quote} \ (a.b)) : (\mathbf{Code} \ \Gamma \ \tau, \mathbf{Code} \ \Gamma \ \sigma)} \text{QCONS}$$

`QUASIQUOTE` fixe Δ , le contexte ambiant, pour toute la vérification du corps cité :

$$\frac{\Delta ; \Gamma \vdash^{n+1} e : \tau}{\Delta \vdash^n (\mathbf{quasiquote} \ e) : \mathbf{Code} \ \Gamma \ \tau} \text{QUASIQUOTE}$$

`UNQUOTE` consulte ce même Δ , sans jamais le recalculer : un symbole ne s'échappe que s'il est réellement lié à l'extérieur de la citation.

$$\frac{\Delta \vdash^n e : \mathbf{Code} \ \Gamma \ \tau}{\Delta ; \Gamma \vdash^{n+1} (\mathbf{unquote} \ e) : \tau} \text{UNQUOTE}$$

`eval` ne produit un type que si le code fourni est clos, c'est-à-dire si le contexte

Γ de l'expression évaluée est inclu dans le contexte Δ de l'évaluation :

$$\frac{\Gamma \vdash a : \text{Code } \Delta \ \tau \quad \Delta \subseteq \Gamma}{\Gamma \vdash (\text{eval } a) : \tau} \text{ EVAL}$$

Un contexte plus riche que nécessaire demeure toujours acceptable : le sous-typage suivant l'exprime au niveau des symboles requis par un terme cité.

$$\frac{\Gamma \subseteq \Gamma'}{\text{Code } \Gamma \ \tau <: \text{Code } \Gamma' \ \tau} \text{ SUBCODE}$$

et s'accompagne, comme à l'accoutumée, de la règle de subsomption :

$$\frac{\Gamma \vdash e : \sigma \quad \sigma <: \sigma'}{\Gamma \vdash e : \sigma'} \text{ SUB}$$

Problème restant. $\lambda(f \ . \ g)$, pour f, g symboles libres, admet deux dérivations concurrentes : par QCONS, le type $(\text{Code } \Gamma \ \alpha, \text{Code } \Gamma \ \beta)$; ou, chaque symbole étant pris pour lui-même plutôt que comme référence à typer sous un contexte, le type brut $\text{Sym } \lambda f \times \text{Sym } \lambda g$. Cette ambiguïté reste, à ce stade, non tranchée.